Проектирование суперагента: путь к более автономной агентной системе. Перевод статьи DeepSeek Harness

2 ноября 2026

Сергей Пименов

Агенты | Разработка | Инфраструктура

Инструменты

Слушать | Aa

Автор оригинала: Юцзя Цяо (Yujia Qiao) и команда DeepSeek Harness Оригинал: Exploring Super-agent Design: Towards a More Autonomous Agent System, статья в X от 1 ноября 2026 года

> От редакции. Это полный перевод статьи, которую опубликовал аккаунт DeepSeek Harness в X. Исходный текст написан на китайском, в X выложен английский машинный перевод, сделанный самой командой DeepSeek Harness. Русский текст переведён с этой английской версии. Схемы и графики взяты из оригинала, надписи на них остались английскими, поэтому под каждым изображением есть перевод.



Обложка оригинальной статьи

Оригинальная статья написана на китайском. Это машинный перевод.

В статье обсуждаются архитектурные идеи. Это не описание существующих функций DeepSeek Harness.

1. Суперагент

За последний год агенты для программирования (coding agents) и другие агенты, решающие одну задачу, показали: модели умеют пользоваться инструментами и решать сложные задачи, а постоянного присмотра и пошагового вмешательства человека им нужно всё меньше. Модели стали лучше координировать нескольких агентов, поэтому организовывать и планировать работу между сессиями стало практичнее. Люди всё меньше участвуют в низкоуровневом исполнении, и их роль постепенно смещается к решениям более высокого уровня и приёмке результата.

Более сильные модели открыли и новые применения. В последнее время появилось много персональных агентов (personal agents). Они подключаются напрямую к повседневным мессенджерам, календарю, почте и рабочим платформам, следят за лентами и помогают с задачами. Персональные агенты представляют лишь

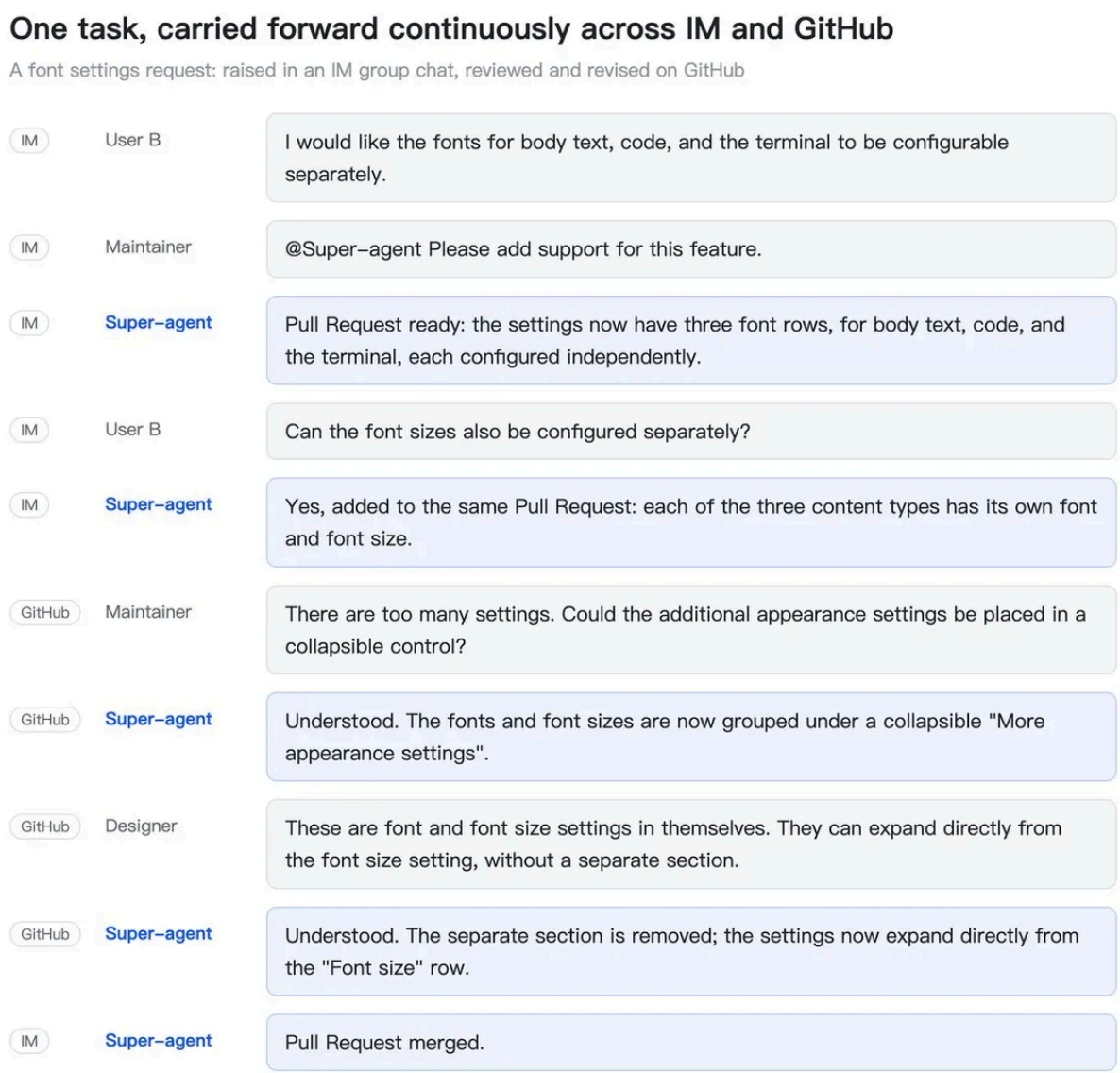
одно из применений такого режима работы, когда агент всегда на связи и реагирует на события. Этот режим не ограничен личным использованием: он ценен и в коммерческой работе, например в разработке программ.

Однако современные распространённые архитектуры агентов с трудом поддерживают такой режим для многих задач сразу или для целой команды. Базовой единицей работы агента по-прежнему остаётся одна сессия, и сессии изолированы друг от друга. Пользователь, который ведёт несколько задач параллельно, вынужден вручную переключаться между сессиями и помнить, что в каждой из них. По нашему опыту, человек обычно способен следить одновременно только за 3–5 активными сессиями. При таких архитектурах одну задачу ограничивают возможности модели, а многозадачность ограничивает внимание человека.

Если добавить к локальному агенту с одной сессией интерфейсы командной строки (CLI), MCP или внешние коннекторы, он сможет получать внешние сообщения, но его базовые ограничения останутся. Он по-прежнему работает в одной синхронной сессии, которая блокируется на долгих операциях, контексты разных задач смешиваются, а при отключении локального устройства всё останавливается. Чтобы такая конструкция продолжала работать, нужно много ручного труда.

Чтобы преодолеть эти ограничения, агента нужно отвязать от любой отдельной сессии, которой управляют вручную. Агент должен стать единой непрерывной сущностью, которая сохраняет контекст между сессиями и работает над несколькими сложными задачами одновременно, причём задачи не мешают друг другу. Такого агента и архитектуру его системы мы называем суперагентом (Super-agent).

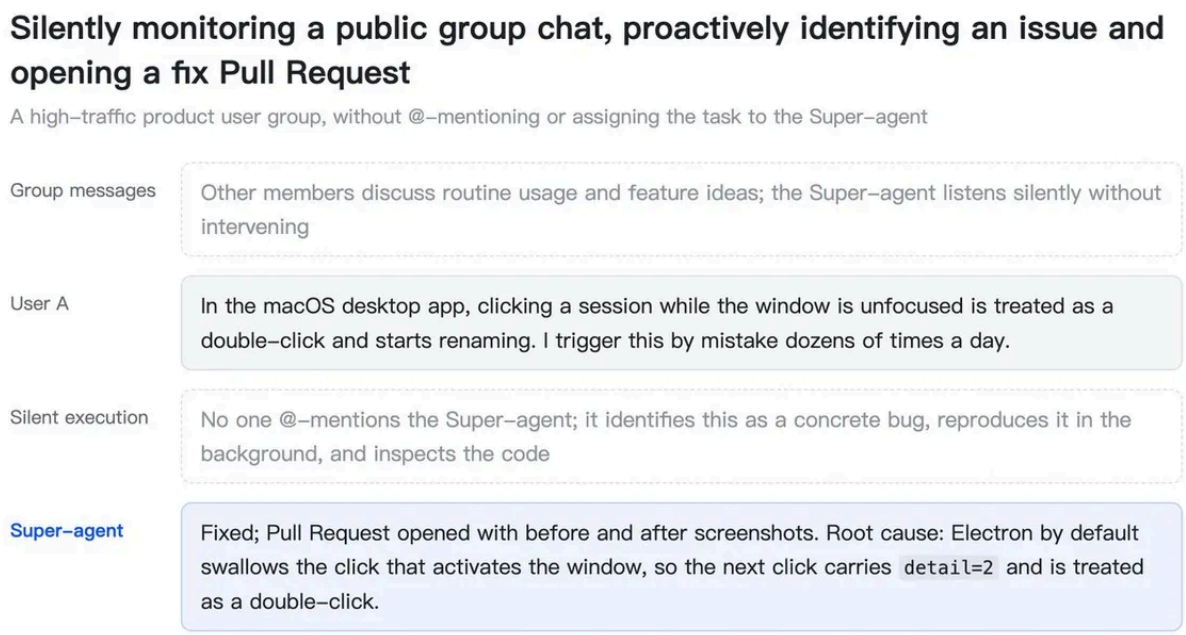
Например, наш внутренний экземпляр суперагента, работающий на DeepSeek Harness, может продолжать одну и ту же задачу разработки в мессенджере (IM) и на GitHub. Он также следит за отзывами в публичных групповых чатах и, даже если его никто не упомянул через @, сам находит первопричину проблемы и открывает пул-реквесты (Pull Request) с исправлением:



Одна задача непрерывно продолжается в мессенджере и на GitHub. Запрос на настройку шрифтов: поднят в групповом чате мессенджера, проверен и доработан на GitHub.

> [IM] Пользователь B: Хочу, чтобы шрифты для основного текста, кода и терминала настраивались отдельно.

[IM] Мейнтейнер: @Super-agent, пожалуйста, добавь поддержку этой функции.
[IM] Суперагент: Пул-реквест готов: в настройках теперь три строки шрифтов, для основного текста, кода и терминала, каждая настраивается независимо.
[IM] Пользователь B: А размеры шрифтов тоже можно настраивать отдельно?
[IM] Суперагент: Да, добавил в тот же пул-реквест: у каждого из трёх типов содержимого свой шрифт и свой размер шрифта.
[GitHub] Мейнтейнер: Настроек слишком много. Можно убрать дополнительные настройки внешнего вида в сворачиваемый элемент?
[GitHub] Суперагент: Понял. Шрифты и размеры шрифтов теперь сгруппированы в сворачиваемом разделе «Дополнительные настройки внешнего вида».
[GitHub] Дизайнер: Это и есть настройки шрифта и его размера. Их можно раскрывать прямо из настройки размера шрифта, без отдельного раздела.
[GitHub] Суперагент: Понял. Отдельный раздел убран, настройки теперь раскрываются прямо из строки «Размер шрифта».
[IM] Суперагент: Пул-реквест слит (merged).



Суперагент молча следит за публичным групповым чатом, сам находит проблему и открывает пул-реквест с исправлением. Многолюдная группа пользователей продукта, суперагента никто не упоминает через @ и не назначает ему задачу.

> Сообщения группы: другие участники обсуждают обычное использование и идеи функций, суперагент молча слушает и не вмешивается.
> Пользователь А: В десктопном приложении для macOS клик по сессии, когда окно не в фокусе, считается двойным кликом и запускает переименование. Я случайно делаю это десятки раз в день.
> Молчаливое выполнение: никто не упоминает суперагента через @, он распознаёт конкретный баг, воспроизводит его в фоне и изучает код.
> Суперагент: Исправлено, пул-реквест открыт со скриншотами до и после. Первопричина: Electron по умолчанию поглощает клик, который активирует окно, поэтому следующий клик приходит с `detail=2` и считается двойным.

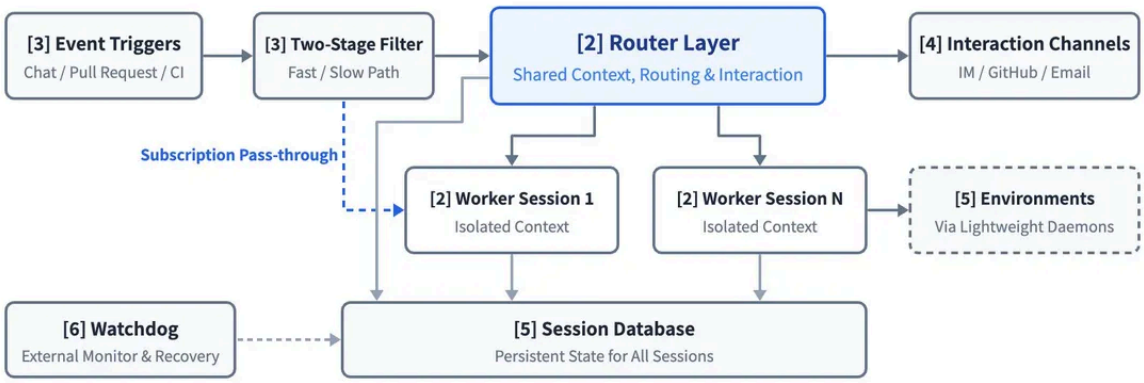
Мы считаем, что у суперагента четыре базовые характеристики:

- **Непрерывность (Continuous).** Пользователи общаются с ним прямо там, где уже работают, например в чатах мессенджера и комментариях на GitHub, и всегда ощущают, что это один агент. Им никогда не приходится гадать, есть ли в контексте сессии X информация Y. Например, если пользователь просит в мессенджере сделать план A, а затем в комментарии к пул-реквесту на GitHub переключается на план B, агент понимает изменение и корректирует свою работу.
- **Продуктивность (Productive).** Он быстро отвечает и параллельно выполняет несколько сложных задач, каждую на высоком уровне. Он может принять следующую инструкцию до того, как завершилась предыдущая команда или задача.
- **Проактивность (Proactive).** Он не ждёт пошаговых инструкций. Он следит за событиями в своём окружении, берётся за всё, что стоит обработать, а в остальных случаях молчит.
- **Постоянство (Persistent).** Он всегда на связи, напрямую подключён к реальным рабочим окружениям пользователя и сам восстанавливается после сбоев при длительной работе.

Ни один цикл ReAct (ReAct loop) не даёт все четыре характеристики сразу, поэтому они должны возникать из архитектуры системы вокруг него. Похожие проектные ограничения обычно приводят к похожим архитектурам. Когда цель в том, чтобы система всегда была на связи и параллельно реагировала на события окружения, мы ожидаем, что персональные агенты и агенты для коммерческой работы, например разработки программ, придут к похожим решениям. При создании такой системы возникает и много задач, очень похожих на те, что давно изучают в компьютерных системах, поэтому здесь применимы многие классические проектные идеи из этой области.

Реализации будут разными, но мы считаем, что основу, которая обеспечивает эти характеристики, составляют пять модулей. Двухуровневая архитектура (раздел 2) даёт одновременно непрерывное взаимодействие и продуктивную параллельную работу. Событийная архитектура (раздел 3) делает агента проактивным. Полнодуплексное многоканальное взаимодействие (раздел 4) делает каждый обмен с пользователями непрерывным и продуктивным. Разделение вычислений, состояния и окружений (раздел 5) вместе со сторожевым агентом (Watchdog, раздел 6) держит агента постоянно на связи на разных устройствах и позволяет ему самому восстанавливаться после сбоев.

На схеме ниже показан общий вид архитектуры суперагента. Модули с метками от [2] до [6] разобраны в разделах 2–6:



Общая схема архитектуры суперагента. [3] Триггеры событий: чат, пул-реквест, CI. [3] Двухэтапный фильтр: быстрый и медленный путь. [2] Слой маршрутизатора: общий контекст, маршрутизация и взаимодействие. [4] Каналы взаимодействия: мессенджер, GitHub, почта. [2] Сессии-исполнители 1…N: изолированный контекст. [5] Окружения: через лёгкие демоны. [5] База данных сессий: постоянное состояние всех сессий. [6] Сторожевой агент: внешний мониторинг и восстановление. Синяя пунктирная стрелка: сквозная передача по подписке (Subscription Pass-through).

2. Двухуровневая архитектура

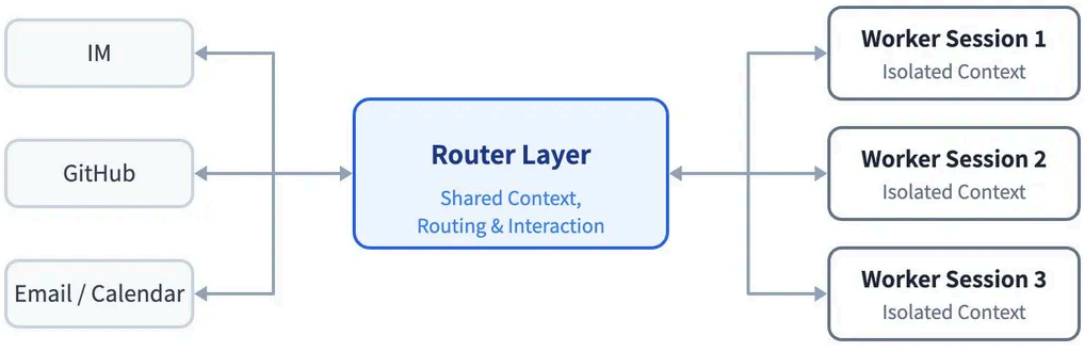
Если нужно одновременно поддерживать непрерывное взаимодействие во всех каналах и продуктивную параллельную

работу над задачами, то в том, как система организует свои сессии, возникает фундаментальный компромисс между непрерывностью и пропускной способностью.

Если направлять все события в одну сессию, контекст будет максимально непрерывным, потому что эта сессия видит всё. Но цикл ReAct выполняет по одному шагу за раз, и это ограничивает скорость одной сессии: задачи выполняются долго, пропускная способность остаётся низкой. Промежуточные детали разных задач к тому же перемешиваются в одном контексте, и результаты становятся хуже.

Если для каждого события создавать новую сессию, компромисс разворачивается в обратную сторону. Параллельная пропускная способность становится максимальной, но когда задачи не полностью независимы, без общего глобального контекста их результаты оказываются несвязанными. А когда несколько сессий отвечают пользователю по отдельности, взаимодействие становится запутанным.

Чтобы получить и непрерывность, и пропускную способность, работу можно разделить между двумя слоями. Долгоживущий слой маршрутизатора (Router layer, раздел 2.1) поддерживает глобальную непрерывность: он ведёт всё взаимодействие с людьми и глобальную координацию. Слой исполнителей (Worker layer, раздел 2.2) с изолированными контекстами выполняет задачи с высокой степенью параллельности, как показано ниже:



Двухуровневая архитектура. Слева каналы: мессенджер, GitHub, почта и календарь. В центре слой маршрутизатора: общий контекст, маршрутизация и взаимодействие. Справа сессии-исполнители 1, 2 и 3, у каждой изолированный контекст.

В двухуровневой архитектуре управление контекстом и параллельное выполнение берёт на себя система, поэтому пользователям редко приходится думать о сессиях, и они всегда работают с одним и тем же суперагентом. Выходит, что заставлять пользователей вручную управлять сессиями во многом привычка проектирования, унаследованная от низкоуровневых API больших языковых моделей. Сессии должны быть деталью реализации системы и не должны ложиться умственной нагрузкой на пользователей.

В существующих архитектурах сессия выполняет две функции: управляет контекстом модели и служит для пользователя единицей управления проектом. Как только продукт отвязан от нижележащих сессий, управление проектами больше не обязано следовать границам сессий. Какое взаимодействие и какое управление задачами предложить пользователям, становится отдельным продуктовым вопросом, к которому мы вернёмся позже.

Два уровня означают логическое разделение обязанностей. Ни один из слоёв не обязан быть реализован как одна сессия, и у

каждого может быть своя внутренняя структура. В следующих двух подразделах слои описаны по очереди.

2.1 Слой маршрутизатора

Слой маршрутизатора (Router) работает как долгоживущий глобальный слой маршрутизации: он принимает взаимодействие с людьми из всех точек входа. Вся межзадачная информация и все решения сходятся здесь, поэтому он служит для системы глобальной точкой упорядочивания, точкой координации и памятью. Чтобы этот единственный глобальный узел не стал узким местом по производительности, слой маршрутизатора спроектирован так, что сам задачи не выполняет. Он только общается с людьми и раздаёт задачи и может вообще работать без инструмента Bash.

Один центральный узел раздаёт задачи нескольким исполняющим узлам, и такая структура похожа на типичную команду агентов (Agent Team), но у них разный фокус. Команда агентов обычно разбивает одну сложную задачу на шаги и координирует роли, которые над ними работают. Эти шаги, как правило, тесно связаны, поэтому ведущему агенту (Team Lead) часто приходится глубоко понимать задачу, планировать подзадачи и часто координировать участников, и в сумме это тяжёлая нагрузка на рассуждения. В суперагенте слой маршрутизатора только раздаёт задачи и отслеживает на высоком уровне, как они связаны между собой. Отдельные задачи он не декомпозирует. И наоборот, команда агентов тоже может работать как двухуровневая архитектура. Для этого её промпты должны запрещать главной сессии выполнять задачи самой и ограничивать её внешним взаимодействием и раздачей отдельных задач участникам, которые работают над ними независимо. По нашему опыту, работа слоя маршрутизатора часто

оказывается проще, чем ожидаешь, и более лёгкая модель обычно обеспечивает и скорость, и качество.

Внутри слой маршрутизатора тоже не обязан быть одной сессией, а части его логики большая языковая модель (LLM) может вообще не понадобиться. Мы выбрали самый прямой вариант, одну долгоживущую сессию LLM, и выяснили, что она становится узким местом по пропускной способности реже, чем может показаться. Причина в том, что события окружения (триггеры из раздела 3), которые приходят во время одного шага вывода (inference), встают в очередь и обрабатываются вместе на следующем шаге. Получается пакетная обработка (batching), и совместная обработка событий почти не добавляет накладных расходов:

- Когда внешние события приходят с постоянной частотой, триггеры попадают в сессию маршрутизатора (Router Session) по одному, и каждый ответ возвращается быстро.
- Когда события приходят всплесками и начинают копиться в очереди сессии маршрутизатора, триггеры, накопившиеся за предыдущий шаг, автоматически объединяются и вместе передаются в следующий вызов LLM. Сессия маршрутизатора выполняет только лёгкое взаимодействие и раздачу задач, поэтому на каждом шаге у неё обычно длинный вход и короткий выход. Триггеры, объединённые в один вызов API LLM, обрабатываются параллельно на этапе предзаполнения (Prefill), поэтому время на один запрос растёт гораздо медленнее размера пакета. Значит, до определённого предела чем больше накопилось, тем меньше амортизированное время вывода на одно решение. Предел зависит от длины контекста и от того, сколько триггеров модель может точно обработать за один шаг.

В разделе 7.2 эффект пакетной обработки измерен в эксперименте. Вместе с подпиской и сквозной передачей (раздел

3.2) и цепочкой вспоминания (Chain of Recall, раздел 3.3) одна долгоживущая сессия маршрутизатора сохраняет непрерывность контекста, и для команды нашего нынешнего размера её уже достаточно.

Если система вырастет настолько, что одна сессия действительно перестанет справляться, слой маршрутизатора может использовать внутри многоэтапную фильтрацию или разделиться на несколько сессий горизонтальным шардированием (sharding). Тогда его внутреннее устройство сводится к тому же компромиссу между непрерывностью и пропускной способностью. Чем мельче шардирование, тем выше параллельная пропускная способность слоя маршрутизатора, но тем меньше памяти и контекста у шардов общие. Этот компромисс останется, пока не решена задача глобальной памяти.

2.2 Слой исполнителей

Слой исполнителей состоит из сессий, которые выполняют задачи. У каждого исполнителя (Worker) изолированный контекст и полный набор инструментов для выполнения, и он занят одной задачей. Сессии-исполнители (Worker Session) напрямую определяют результат каждой задачи и качество её кода, поэтому им обычно нужны самые сильные модели.

Исполнители никогда не общаются с людьми напрямую. Их рассуждения и вызовы инструментов не попадают во внешние каналы. Вместо этого каждый исполнитель сообщает о своём состоянии слою маршрутизатора, который ведёт все ответы пользователю. Так система сохраняет и параллельность, и качество результатов и избегает путаницы, когда несколько сессий отвечают по отдельности.

Когда задачи выполняются параллельно, каждый исполнитель работает над своей задачей, и равноправным исполнителям

обычно не нужно общаться напрямую. Конфликты между исполнителями чаще всего возникают в общем рабочем окружении, например в одном и том же пул-реквесте. Если все вовлечённые исполнители подписаны на тред (Thread) этого окружения, они получают одни и те же уведомления о событиях (раздел 3.2) и могут сами заметить и разрешить конфликт. Немногие случаи, которые требуют координации между задачами, передаёт слой маршрутизатора.

Как и слой маршрутизатора, исполнитель не обязан внутри быть одной сессией. Для сложной задачи исполнитель может создавать субагентов (subagent) или сам быть командой агентов, и для слоя маршрутизатора над ним это прозрачно. Та же структура подходит для совместной работы над кодом, где независимые задачи, например воспроизведение проблем, ревью пул-реквестов и разработка функций, могут идти одновременно и не блокировать друг друга. Она подходит и для персональных агентов, хотя персональный агент обычно предлагает только одну страницу чата. Когда пользователь обсуждает в этом чате несколько тем сразу, страница сохраняет непрерывность разговора. Тем временем бэкенд может выделить отдельного исполнителя под каждую тему, чтобы параллельно работать над задачами и готовить черновики ответов.

3. Событийные триггеры

Чтобы суперагент стал проактивным, сначала нужно изменить то, что приводит систему в движение. Основной вход агента теперь не промпты, которые человек по одному вводит в сессию, а полуструктурированные события из окружения. В суперагенте такие события окружения, например сообщения в групповых чатах, письма, напоминания календаря, комментарии ревью кода и результаты CI, называются триггерами (Trigger). Агент слушает триггеры и реагирует на них, поэтому ему больше не нужно ждать

команды человека перед каждым запуском, как агенту с одной сессией.

Подразделы ниже прослеживают событие окружения от поступления до обработки в четыре шага, каждый из которых опирается на предыдущий. Сначала двухэтапные политики, сочетающие код и модели, отсеивают большой объём шума (раздел 3.1). Затем механизм подписки и сквозной передачи направляет события исполнителям (раздел 3.2). После этого встраивание и вытеснение (Inline & Spill) и цепочка вспоминания передают в модель плотный контекст и по запросу поднимают дополнительный (раздел 3.3). Наконец, спекулятивное выполнение позволяет агенту действовать на опережение в фоне по сигналам окружения (раздел 3.4).

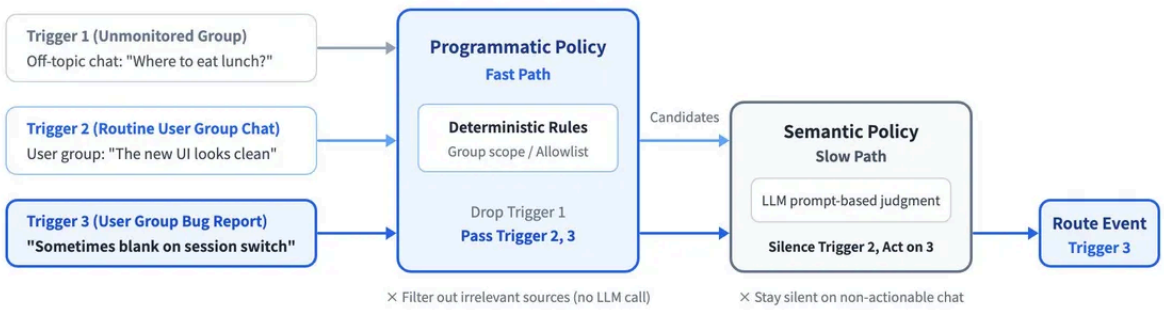
3.1 Фильтрация событий

После подключения к реальным групповым чатам и платформам для кода система видит шумный поток событий с низкой долей срабатываний: подавляющее большинство повседневных сообщений не требует от агента никаких действий. Если большая модель будет с нуля оценивать каждое событие, это добавит слишком много задержки и потребует слишком много вычислений. Мы заимствуем из компьютерных систем идею быстрого и медленного пути (Fast Path / Slow Path) и считаем рассуждения большой модели гибким, но дорогим медленным путём. Частые, чётко определённые и стабильные решения можно превратить в детерминированный код, который служит быстрым путём с низкой задержкой и низкой стоимостью.

Поэтому система сначала фильтрует каждое входящее событие. Детерминированные политики убирают шум ещё до вызова большой модели, и только события-кандидаты, которым нужно смысловое понимание, попадают в модель. Мы используем

двухэтапную фильтрацию, которая сочетает программные политики с большой моделью:

- **Программные политики (быстрый путь).** Код выполняет быструю проверку допуска по детерминированным условиям: источнику события, группе, разрешённым спискам (allowlist) пользователей или ботов. Например, для каждого источника можно задать режим «отвечать только на явные запросы», «считать событиями-кандидатами» или «игнорировать». Эти проверки отбрасывают большинство нерелевантных сообщений ещё до вызова модели.
- **Смысловые политики (медленный путь).** События-кандидаты, прошедшие программные проверки, попадают в LLM, которая решает, действовать ли, на основе контекста и политик, записанных в виде промптов. Смысловой этап служит и естественной точкой расширения на обычном языке: люди в любой момент могут включать и выключать политики или добавлять новые через промпты, как показано ниже:

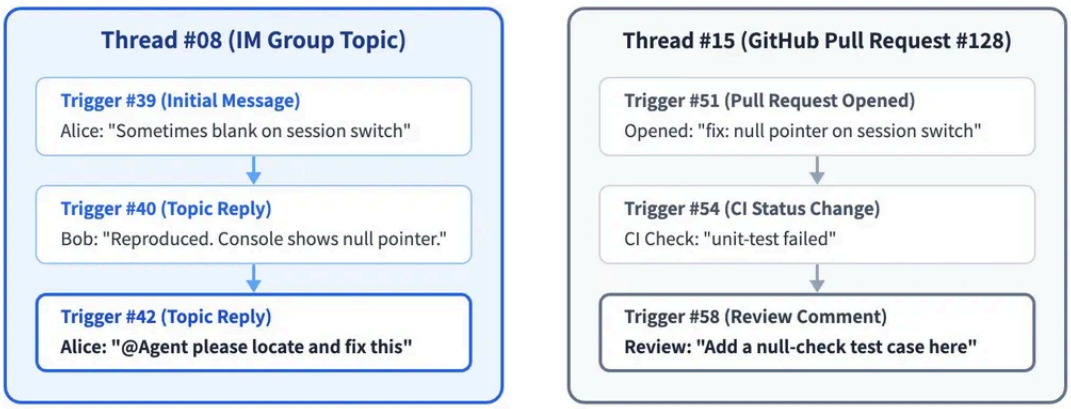


Двухэтапная фильтрация. Триггер 1 (группа без мониторинга): разговор не по теме, «Где пообедать?». Триггер 2 (обычный чат группы пользователей): «Новый интерфейс выглядит аккуратно». Триггер 3 (сообщение о баге в группе пользователей): «Иногда пустой экран при переключении сессии». Программная политика, быстрый путь: детерминированные правила (охват групп, разрешённый список); триггер 1 отброшен, триггеры 2 и 3 пропущены. Отсев нерелевантных источников без вызова LLM. Смысловая политика, медленный путь: оценка LLM по промпту; на триггер 2 молчание, на триггер 3 действие. Молчание в ответ на чат, где делать нечего. Маршрутизация события: триггер 3.

3.2 Маршрутизация событий

Глобальный слой маршрутизатора из раздела 2 сохраняет непрерывность взаимодействия во всех каналах. Но если бы каждое последующее событие по уже назначенной задаче всё равно проходило через слой маршрутизатора, этот единственный глобальный узел нёс бы большую нагрузку и замедлял ответы. Поэтому разделение на быстрый и медленный путь применяется и к маршрутизации отфильтрованных событий. Слой маршрутизатора занимается только холодным стартом новых задач и координацией между задачами, а последующие события код отправляет напрямую в сессию-исполнителя, которой они принадлежат.

В реальной работе последовательные события в одном пул-реквесте на GitHub, в одной теме мессенджера или в одной цепочке писем образуют поток событий, который мы называем тредом (Thread):



Примеры тредов. Тред #08 (тема в группе мессенджера): триггер #39 (первое сообщение), Алиса: «Иногда пустой экран при переключении сессии»; триггер #40 (ответ в теме), Боб: «Воспроизвёл. В консоли null pointer»; триггер #42 (ответ в теме), Алиса: «@Agent, найди и исправь, пожалуйста». Тред #15 (пул-реквест #128 на GitHub): триггер #51 (пул-реквест открыт): «fix: null pointer on session switch»; триггер #54 (изменение статуса CI), проверка CI: «unit-test failed»; триггер #58 (комментарий ревью): «Добавь сюда тест на проверку null».

Когда слой маршрутизатора создаёт сессию-исполнителя для первого события в треде, этот исполнитель подписывается на тред напрямую. С этого момента маршрутизация событий идёт по одному из двух путей, показанных ниже:

- **С подпиской (быстрый путь).** Код пропускает последующие события треда напрямую к подписанному исполнителю, и слою маршрутизатора не нужен дополнительный вызов инструмента, чтобы их переслать. Если новое событие выходит за рамки текущей задачи, исполнитель может отписаться или вернуть событие слою маршрутизатора для повторного распределения.
- **Без подписки (медленный путь).** Слой маршрутизатора по-прежнему видит каждое событие и благодаря этому отслеживает глобальное состояние, но оценивает и распределяет только новые события, у которых ещё нет подписчика.



Два пути маршрутизации. События в треде (например, один и тот же пул-реквест или тема мессенджера). Первое событие (медленный путь) идёт в слой маршрутизатора, который создаёт исполнителя и подписку. Последующие события идут сквозной передачей (быстрый путь) прямо в сессию-исполнителя, подписанную на тред.

Подписки между тредами и исполнителями устроены по схеме «многие ко многим». Исполнитель может подписаться на несколько связанных тредов, а у треда может быть несколько подписанных исполнителей. Когда исполнители пересекаются в

одном рабочем окружении или обсуждении, каждое новое событие в общем треде сразу проходит ко всем его подписчикам. Тогда исполнители могут узнать, чем заняты другие, и сами разрешить конфликты без прямого общения друг с другом. На выходе исполнители по-прежнему сообщают ключевые выводы слою маршрутизатора, который ведёт все ответы пользователю и синхронизирует глобальную память.

3.3 Организация контекста

После того как событие отфильтровано и направлено в целевую сессию, возникает следующий вопрос: сколько информации поместить в контекстное окно модели. На входе теперь не ограниченное число промптов от человека, а полуструктурированные события, которые окружение порождает непрерывно, поэтому устройство вызовов инструментов и передачи событий становится важнее. Если передать слишком много, окно заполнится и внимание модели рассеется. Если слишком мало, модели придётся снова и снова вызывать инструменты, чтобы прочитать контекст, и шагов выполнения станет больше. Тратить контекстное окно впустую значит тратить впустую интеллект модели.

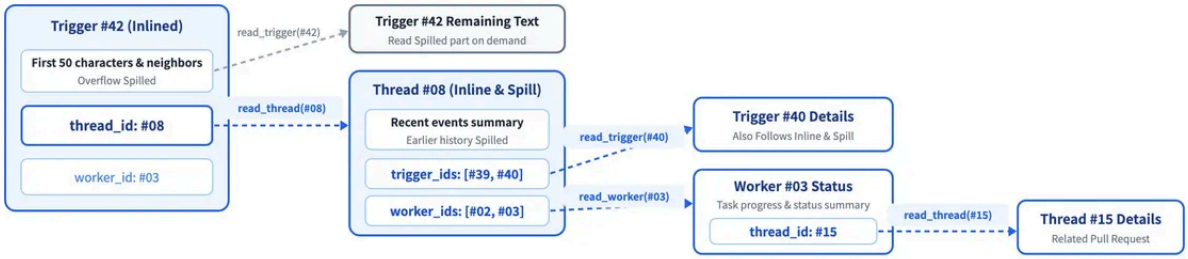
Поэтому ограниченное контекстное окно можно рассматривать как кэш. Если пользоваться терминами из оптимизации компиляторов и компьютерных систем, передача контекста сводится к двум базовым операциям. Встраивание (Inline) помещает информацию, которая скорее всего скоро понадобится, прямо в текущее контекстное окно и экономит лишние циклы вызова инструментов. Вытеснение (Spill) держит полную информацию за пределами окна, когда она превышает бюджет или нужна редко, и оставляет в окне только ссылку или краткое содержание, по которым модель может прочитать полную информацию по запросу.

В сочетании с локальностью (locality), классической идеей проектирования компьютерных систем, встраивание и вытеснение позволяют пошагово улучшать передачу событий и вызовы инструментов:

- **Начать с минимума информации.** Сначала дать модели только ID текущего объекта и краткое содержание.
- **Постепенно встраивать связанную информацию по принципу локальности.** Встраивать то, что модели, скорее всего, скоро понадобится. Начать с ID тесно связанных объектов, затем добавить содержимое, соседнее в пространстве или во времени. Примеры: первые 50 символов содержимого, сообщения мессенджера вокруг данного сообщения и сообщение, на которое оно отвечает, соседние комментарии в том же обсуждении ревью кода.
- **Вытеснять содержимое сверх лимита.** Для длинного содержимого, которое нельзя встроить целиком, задать небольшой лимит длины и вытеснить остальное за пределы окна. Простой способ: обрезать содержимое и оставить указатель на место, где оно продолжается, чтобы модель могла вызвать инструмент и дочитать остальное, когда понадобится.

У слоя маршрутизатора своя проблема с контекстом. Многие последующие события напрямую уходят к исполнителям, а прошлые задачи накапливаются за время долгой работы, поэтому слой маршрутизатора не может держать полную историю каждой задачи в своём контекстном окне. Но когда пользователь вспоминает старую задачу в другом канале, слою маршрутизатора всё равно нужно восстановить её контекст. Если последовательно применить встраивание и вытеснение ко всем видам объектов в системе, получается механизм многошагового извлечения по запросу: цепочка вспоминания (Chain of Recall). Она уменьшает

потерю ранней информации слоем маршрутизатора при долгой работе, как показано ниже:



Цепочка вспоминания. Триггер #42 (встроен): первые 50 символов и соседние сообщения, переполнение вытеснено; thread_id: #08; worker_id: #03. Вызов read_trigger(#42) читает оставшийся текст триггера #42, то есть вытесненную часть по запросу. Вызов read_thread(#08) открывает тред #08 (встраивание и вытеснение): краткое содержание последних событий, более ранняя история вытеснена; trigger_ids: [#39, #40]; worker_ids: [#02, #03]. Вызов read_trigger(#40) даёт подробности триггера #40, к ним тоже применяется встраивание и вытеснение. Вызов read_worker(#03) даёт статус исполнителя #03: краткое описание хода задачи и состояния; thread_id: #15. Вызов read_thread(#15) даёт подробности треда #15: связанный пул-реквест.

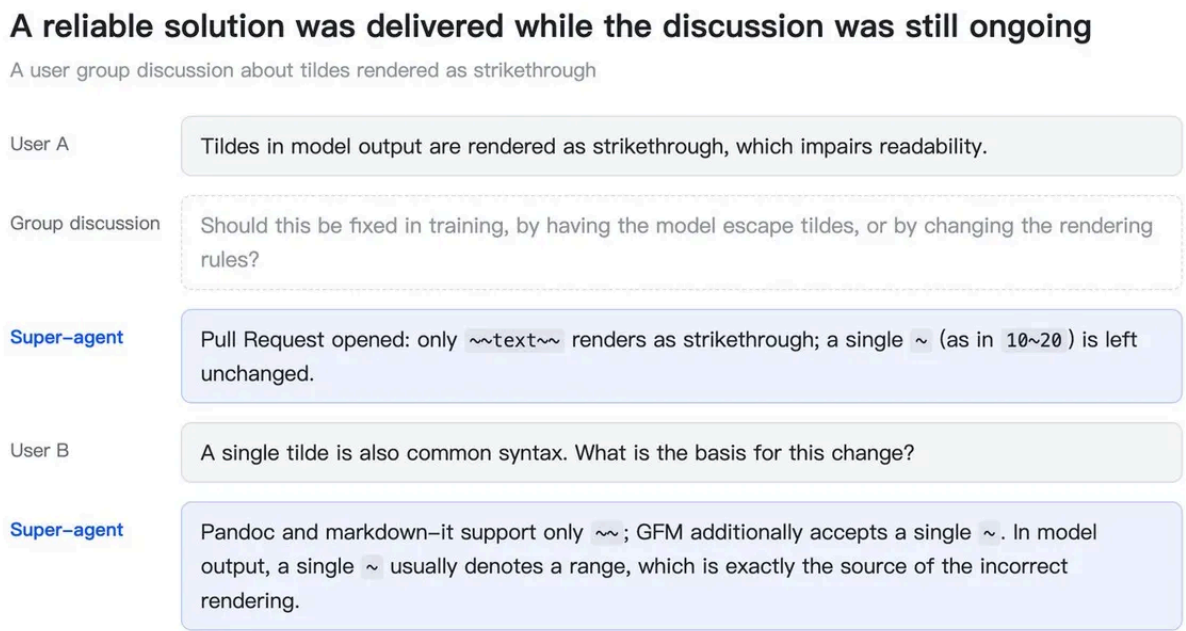
Система присваивает уникальные ID таким объектам, как триггеры, треды и сессии-исполнители, и при передаче одного объекта встраивает ID тесно связанных с ним объектов. Эти ID связывают объекты в граф ссылок. Каждый объект, поднятый через вызов инструмента, тоже подчиняется правилу встраивания и вытеснения, поэтому возвращённый результат встраивает краткое содержание и ID объектов следующего шага. Начав с текущего события, модель может идти по этим ссылкам и поднимать контекст шаг за шагом настолько далеко, насколько нужно.

Например, модель может начать с треда текущего сообщения, по встроенному ID перейти к связанной сессии-исполнителю, а затем найти другой тред ревью кода, связанный с этим исполнителем.

3.4 Действия на опережение

Мы также заимствуем из компьютерных систем спекулятивное выполнение (speculative execution). По сигналам окружения агент начинает работу в фоне, ещё не зная, пригодится ли результат. Он тратит дополнительные фоновые вычисления, чтобы людям меньше приходилось ждать и реже вмешиваться. Результаты, которые оказались полезными, он передаёт, остальные отбрасывает.

В сценариях персонального агента, когда приходит письмо с приглашением на встречу или сообщение с задачей, агент может в фоне проверить конфликты в календаре, собрать справочную информацию и подготовить черновик ответа. Пользователю остаётся просмотреть черновик и отправить его. Тот же подход работает в совместной разработке. Например, в нашей внутренней группе пользователей DeepSeek Harness мы настроили политику «молча воспроизвести и исправить». Когда кто-то сообщает о проблеме или обсуждает предлагаемое изменение, агент сначала пытается в фоне воспроизвести проблему и найти исправление. Результат он передаёт, только когда подтвердил проблему и нашёл простое исправление. В противном случае он всё время молчит.



Надёжное решение готово, пока обсуждение ещё шло. Обсуждение в группе пользователей: тильды отображаются как зачёркивание.

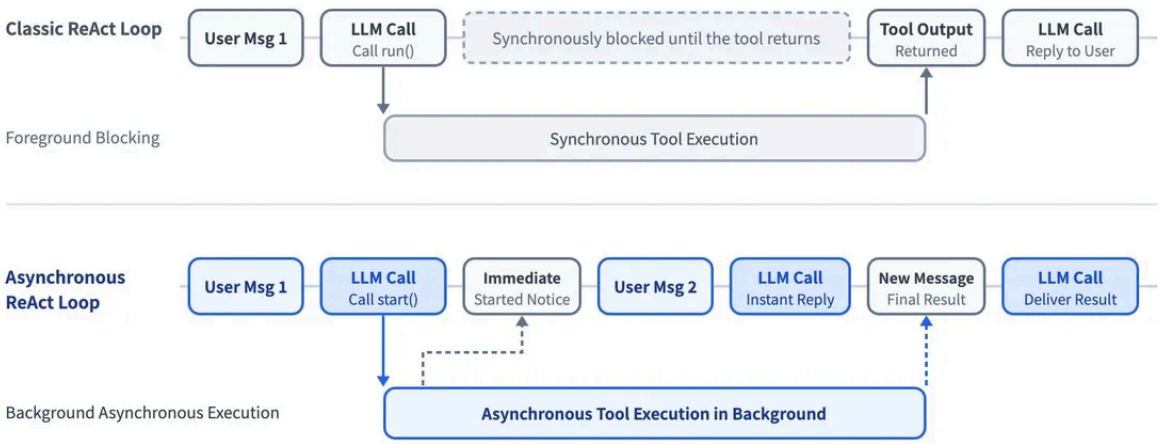
> Пользователь A: Тильды в выводе модели отображаются как зачёркнутый текст, читать неудобно.
> Обсуждение в группе: Исправлять это при обучении, научив модель экранировать тильды, или изменить правила отображения?
> Суперагент: Пул-реквест открыт: зачёркиванием отображается только `~~text~~`, одиночная ~ (как в 10~20) остаётся без изменений.
> Пользователь B: Одиночная тильда тоже распространена как синтаксис. На чём основано это изменение?
> Суперагент: Pandoc и markdown-it поддерживают только ~~; GFM дополнительно принимает одиночную ~. В выводе модели одиночная ~ обычно обозначает диапазон, и именно отсюда неправильное отображение.

4. Полнодуплексное многоканальное взаимодействие

Двухуровневая архитектура из раздела 2 сохраняет непрерывность взаимодействия во всех каналах на уровне контекста. Но чтобы повседневное взаимодействие было и непрерывным, и продуктивным, нужно решить ещё две задачи: как устроено взаимодействие и где оно происходит. Первая задача: не дать долгим фоновым операциям блокировать разговор на переднем плане. Её решает асинхронный цикл ReAct, и сессия может в любой момент принимать новые инструкции, не дожидаясь завершения предыдущей команды или задачи (раздел 4.1). Вторая задача: подстраивать содержание и подробность ответов агента под каждый канал связи и под то, как в нём общаются люди (раздел 4.2).

4.1 Полный дуплекс

В классическом цикле ReAct, когда модель делает синхронный вызов инструмента run(), цикл блокируется, пока инструмент не завершится и не вернёт итоговый результат. До этого момента сессия не может реагировать ни на что новое от пользователя. Чтобы убрать эту блокировку, разговор нужно перевести из пошагового полудуплексного (half-duplex) режима в полнодуплексный (full-duplex), как показано ниже:



Классический и асинхронный цикл ReAct. Классический цикл: сообщение пользователя 1, вызов LLM (вызов run()), синхронная блокировка до возврата инструмента, вывод инструмента возвращён, вызов LLM (ответ пользователю); на переднем плане блокировка, пока идёт синхронное выполнение инструмента. Асинхронный цикл: сообщение пользователя 1, вызов LLM (вызов start()), немедленное уведомление о запуске, сообщение пользователя 2, вызов LLM (мгновенный ответ), новое сообщение (итоговый результат), вызов LLM (передача результата); асинхронное выполнение инструмента идёт в фоне.

Для этого мы заменяем долгие вызовы команд и инструментов асинхронными вызовами start() и stop() и превращаем цикл в асинхронный цикл ReAct. Вызов start() сразу возвращает «Started», и цикл не ждёт инструмент. Когда инструмент завершается в фоне, система передаёт его итоговый результат в сессию как новое сообщение-событие. По контексту модель может также вызвать stop(), чтобы завершить фоновую команду, которая работает слишком долго.

Когда разговор на переднем плане отвязан от фоновых задач, агент может отвечать людям, пока долгие задачи идут в фоне. Новый ввод обрабатывается, как только заканчивается текущий шаг вывода, и не ждёт, пока завершатся фоновые инструменты.

DeepSeek Harness уже реализует этот механизм асинхронных задач, но по умолчанию он пока не включён.

4.2 Многоканальность

В групповых чатах, почте и обсуждениях на GitHub люди общаются по-разному. Агенту нужно подстраивать и подробность, и манеру ответа под каждый канал. Подробные рассуждения и журналы вызовов инструментов могут подходить для терминала с одним пользователем, но если без изменений публиковать их в групповом чате, участники утонут в информации.

В суперагенте часть этой задачи уже решает двухуровневая архитектура. Подробные рассуждения и частые вызовы инструментов каждой задачи остаются внутри её сессии-исполнителя и никогда не попадают напрямую во внешний канал, а всё внешнее взаимодействие ведёт слой маршрутизатора. Помимо этого, системе нужно подключиться к рабочим окружениям, которыми пользователи пользуются каждый день, и с помощью промптов задать нормы общения для каждого канала:

- **Мессенджеры и другие каналы мгновенных сообщений.** Общаться коротко и как можно реже отвлекать людей. Как человек в команде, сообщать людям только тогда, когда нужно решение или приёмка, задавать только прямые вопросы и давать короткие выводы. Одно простое и эффективное жёсткое правило, которое мы проверили: не больше 100 слов в ответе.
- **Асинхронные каналы: GitHub, почта, документы.** Давать структурированный и плотный по информации результат.

Например, при создании или обновлении пул-реквеста полностью описывать предысторию проблемы, суть изменения и результаты проверки по правилам проекта. В сценариях персонального агента сразу готовить черновики структурированных писем и краткие изложения документов.

5. Разделение вычислений, состояния и окружений

По умолчанию локальные агенты привязывают к одному устройству три вещи: вычисления, на которых работает цикл агента (Agent Loop); состояние сессии, которое должно сохраняться; и окружение выполнения, например репозитории кода и локальные наборы инструментов. Агент останавливается, когда устройство засыпает или уходит в офлайн, и не может дотянуться до окружений на других устройствах. Чтобы суперагент был постоянно на связи и подключён к реальным рабочим окружениям, которые нужны его задачам, мы заимствуем из компьютерных систем разделение вычислений и хранения (compute-storage separation) и разделяем эти три вещи в два шага. Сначала мы отделяем вычисления от состояния сессии: так агент освобождается от жизненного цикла любой отдельной машины и постоянно работает в облаке (раздел 5.1). Затем мы отвязываем облачный цикл агента от окружений выполнения, в которых много состояния: они остаются на своих местах, а агент обращается к ним с разных устройств (раздел 5.2).

5.1 Отделение вычислений от состояния

Вычисления, на которых работает цикл агента, могут не хранить состояния, сохраняться должно только состояние сессии. Поэтому первый уровень разделения запускает эти вычисления в облачных контейнерах без состояния (stateless) с эластичным масштабированием, а сессии и другое постоянное состояние

хранит в базе данных. Контейнеры можно масштабировать, перезапускать или заменять, а постоянное хранилище сохраняет непрерывность сессий и задач. Локальное устройство, которое засыпает или уходит в офлайн, больше не останавливает агента: он постоянно работает в облаке.

5.2 Отделение вычислений от окружений выполнения

Когда цикл агента и состояние сессии в облаке, следующий вопрос в том, как агент получает доступ к окружениям выполнения, которые нужны его задачам. Значительная часть контекста, с которым работает модель, приходит из вызовов инструментов к репозиториям кода, локальным файлам, сервисам-зависимостям и другим окружениям выполнения. Значит, в широком смысле контекст включает и само окружение выполнения.

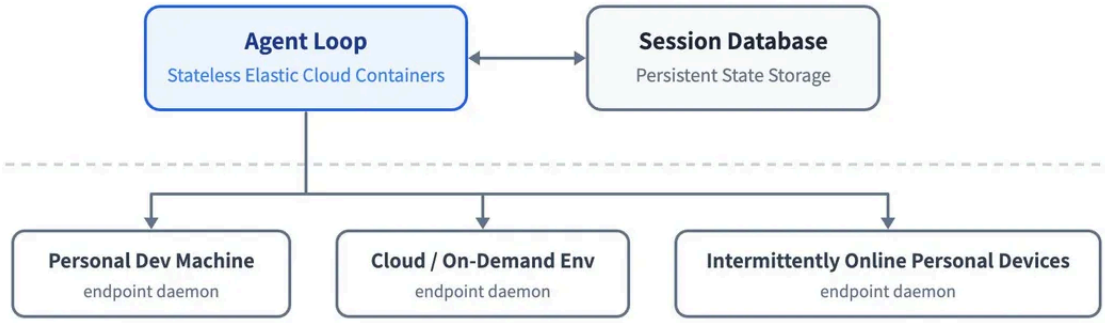
Есть два распространённых подхода. Первый запускает песочницу (sandbox) в облаке, загружает код и пытается воссоздать и синхронизировать окружение. Но реальные рабочие окружения часто содержат незакоммиченные локальные изменения, личные учётные данные, зависимости от сервисов во внутренней сети и определённое оборудование или наборы инструментов. Синхронизировать всё это дорого, и легко что-то упустить. Второй подход предлагает пользователям отказаться от локальных окружений и перенести работу в облако, а для многих пользователей такая миграция слишком дорога.

Трудно копировать и синхронизировать само окружение, поэтому переносить его в облако вместе с циклом агента не нужно. Второй уровень разделения запускает цикл агента в облаке и оставляет окружения выполнения, в которых много состояния, на своих местах. На каждой конечной точке нужен только лёгкий демон (daemon), который не хранит состояния сессии и только

выполняет команды и возвращает результаты. Окружение выполнения остаётся вне облака, и агент читает из него контекст по запросу.

Тот же подход описан в статье Anthropic «Decoupling the brain from the hands» («Отделить мозг от рук»): обвязка агента (harness) не хранит состояния, журналы сессий хранятся вне её, а окружения выполнения подключаются по запросу через единый интерфейс. Разница в целевом сценарии. Статья Anthropic описывает управляемый сервис и в основном обсуждает песочницы, создаваемые по запросу, и ресурсы внутри виртуальных частных облаков (VPC) клиентов. Нам в первую очередь нужно подключать личные машины, которыми люди пользуются каждый день: на них лежат незакоммиченные изменения и локальные учётные данные, и они могут быть в сети лишь время от времени. Поэтому наша архитектура поддерживает BYOM (Bring Your Own Machine, «своя машина»), то есть подключение собственной машины разработчика или личного компьютера пользователя к облачному агенту. Она поддерживает и облачные машины, и вычислительные окружения, создаваемые по запросу.

В результате агент больше не привязан к одной машине. Он может переключаться между устройствами выполнения и запускать команды и читать контекст в том окружении, которое нужно задаче. Он может работать даже с устройствами, которые бывают в сети лишь время от времени: когда личное устройство пользователя снова подключается, облачный агент автоматически запускает и завершает задачи, которым нужно это устройство. На схеме ниже показана итоговая структура, где вычисления, постоянное состояние и окружения выполнения разделены на двух уровнях:



Разделение вычислений, состояния и окружений. Цикл агента: облачные контейнеры без состояния с эластичным масштабированием. База данных сессий: хранилище постоянного состояния. Ниже три типа окружений, на каждом демон конечной точки: личная машина разработчика; облачное окружение или окружение по запросу; личные устройства, которые бывают в сети время от времени.

6. Сторожевой агент

Чтобы оставаться постоянно на связи, разделения вычислений и хранения из раздела 5 мало: система должна ещё и автоматически восстанавливаться после сбоев при длительной работе. Распределённая система, которая постоянно на связи, редко долго остаётся надёжной, если полагается только на свою внутреннюю логику. Ей нужен независимый контур управления (Control Loop) вне основной системы.

Агентным системам к тому же мало традиционных проверок жизнеспособности процессов (liveness probes) или скриптов мониторинга. Многие сбои не выглядят как падения: процесс работает, но модель застряла в бесполезном цикле или ушла от цели. Поэтому долго работающая агентная система должна следить и за ходом задач, и за жизнеспособностью инфраструктуры.

Роль внешнего контура управления у нас выполняет независимая сессия сторожевого агента (Watchdog Session). Она работает вне основной системы и запускается сообщениями по расписанию. Сторожевой агент разбирается со сбоями инфраструктуры,

например ошибками API и неудачным восстановлением состояния. Он также с помощью модели оценивает, продвигается ли каждый исполнитель по существу или отклонился от общей цели. Он оперативно исправляет сбои и продолжает перепроверять временные внешние сбои, пока они не будут устранены.

Сторожевой агент сам представляет собой агентную сессию с полным набором инструментов, поэтому мейнтейнеры могут на обычном языке спросить его, почему что-то сломалось. Они также могут поручить ему корректировать поведение основной системы прямо во время работы, например добавлять или удалять промпты или настраивать правила триггеров.

7. Предварительные эксперименты

Мы провели предварительные эксперименты с ключевыми механизмами планирования и маршрутизации на двух уровнях: для всей системы и отдельно для слоя маршрутизатора. Сквозной (end-to-end) эксперимент сравнивает двухуровневую архитектуру с архитектурами на одной сессии и измеряет эффект механизма подписки и сквозной передачи (раздел 7.1). Второй эксперимент проверяет пакетную обработку в слое маршрутизатора (раздел 7.2). В экспериментах использованы синтетические нагрузки и упрощённые окружения, а часть постановки могла бы быть строже, поэтому результаты ниже носят ориентировочный характер.

7.1 Абляционное исследование

Четыре сквозные архитектуры работали одновременно на одном потоке сообщений из группового чата и CI на GitHub. В каждом прогоне было около 300 сообщений, из них около 20 были рабочими запросами. Остальное составляли болтовня, уточнения, отмены и сообщения CI. Работу над кодом имитировал

инструмент, который просто тратит время, поэтому эксперимент измеряет планирование и координацию в системе, а не умение выполнять отдельные задачи. Каждую архитектуру запускали 5 раз.

Четыре архитектуры: суперагент (Super-agent), суперагент без сквозной передачи (Super-agent without Pass-through) и две версии одной сессии с инструментом субагентов (Single Session with Subagent tool) с разными промптами.

- Нейтральный промпт: «Use a subagent for a piece of work when appropriate...» («Используй субагента для части работы, когда это уместно...»).
- Промпт с приоритетом делегирования: «Every message of the chat reaches you, so work you run yourself keeps stopping for new messages. Prefer handing each piece of work to a subagent of its own, and do only small things yourself...» («До тебя доходит каждое сообщение чата, поэтому работа, которую ты выполняешь сам, постоянно прерывается новыми сообщениями. Предпочитай отдавать каждую часть работы отдельному субагенту, а сам делай только мелочи...»).

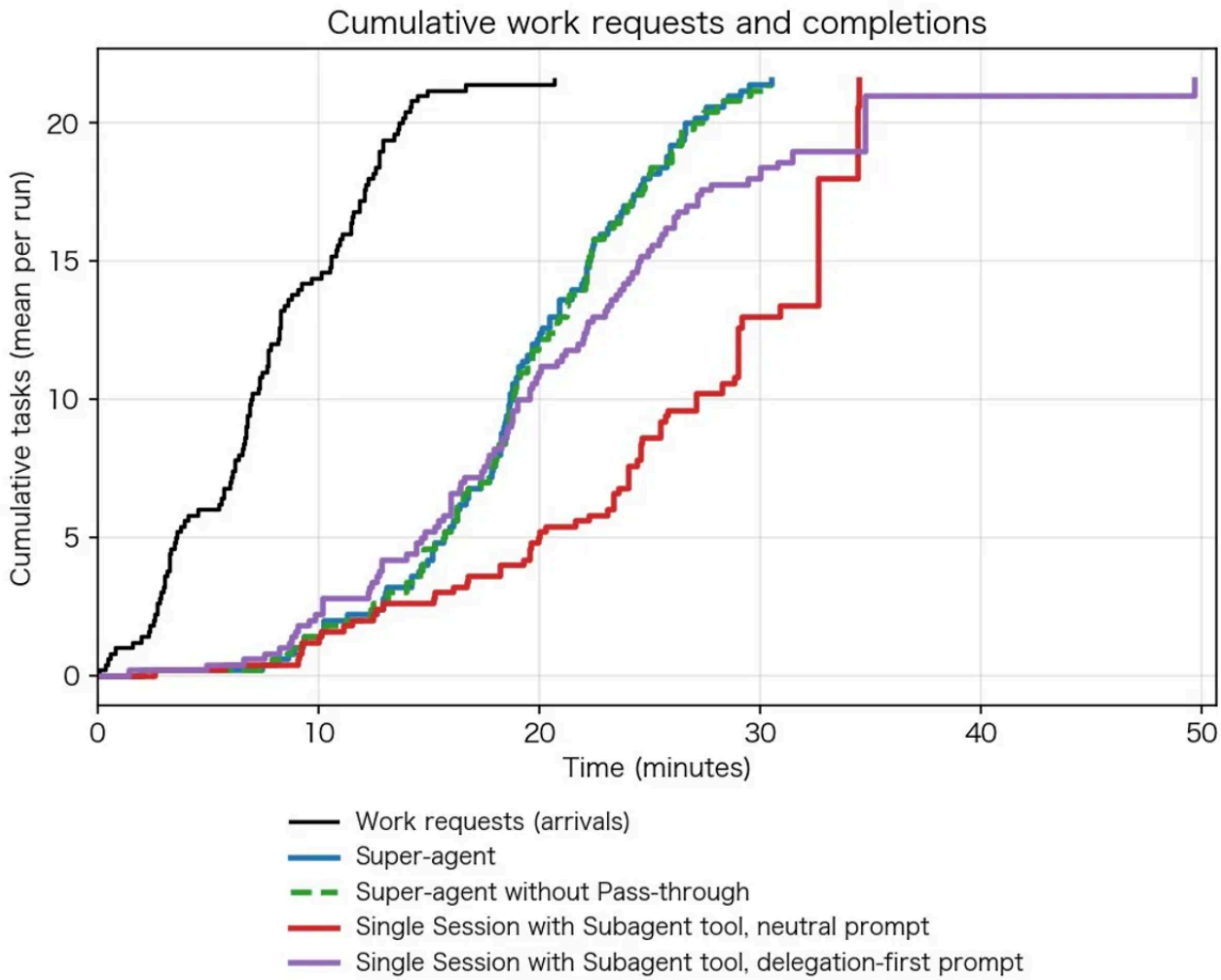


График «Накопленные рабочие запросы и выполненные задачи». По горизонтали время в минутах, по вертикали накопленное число задач (среднее за прогон). Чёрная линия: рабочие запросы (поступление). Синяя: суперагент. Зелёная пунктирная: суперагент без сквозной передачи. Красная: одна сессия с инструментом субагентов, нейтральный промпт. Фиолетовая: одна сессия с инструментом субагентов, промпт с приоритетом делегирования.

Архитектура	**Время выполнения задачи p50 / p95 (мин)**	**Токены, прочитанные из кэша, за прогон (все сессии)**
Суперагент (Super-agent)	12,4 / 15,7	23,2 млн
Суперагент без сквозной передачи (Super-agent without Pass-through)	12,6 / 15,5	31,8 млн
Одна сессия с инструментом субагентов, нейтральный	20,4 / 29,5	6,1 млн

промпт (Single Session with Subagent tool, neutral prompt)		
Одна сессия с инструментом субагентов, промпт с приоритетом делегирования (Single Session with Subagent tool, delegation prompt)	13,8 / 29,0	10,8 млн

Примечание: каждую архитектуру запускали 5 раз. Время выполнения задачи измеряется от рабочего запроса до сообщения с результатом.

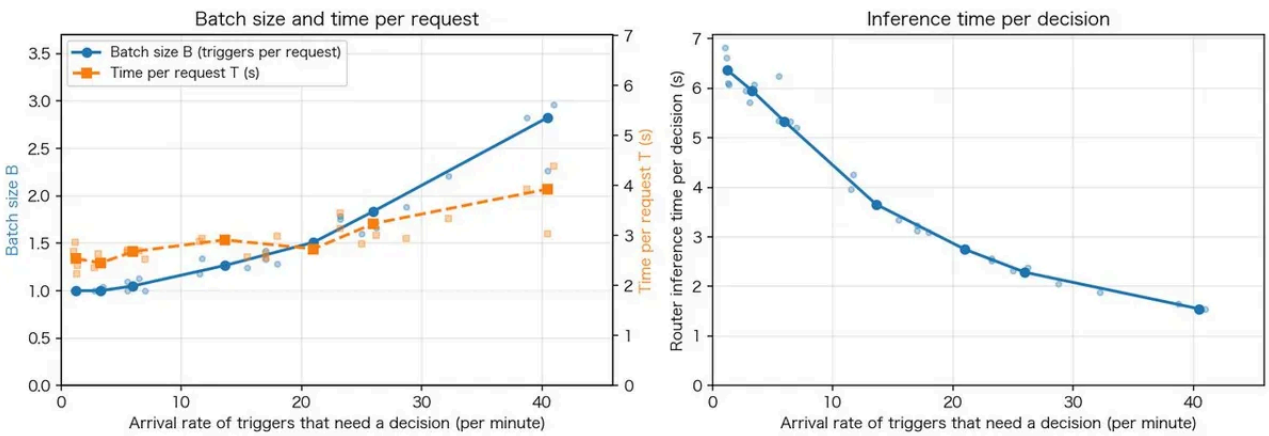
Суперагент выполняет задачи за 12,4 минуты по p50 и за 15,7 минуты по p95, заметно быстрее, чем одна сессия с инструментом субагентов. Архитектура с одной сессией медленнее, потому что в этом эксперименте она сильно зависит от промпта: главная сессия часто решает сделать работу сама, и время выполнения становится долгим и нестабильным. С нейтральным промптом модель не вызвала субагента ни в одном из 5 прогонов. Вся работа шла в главной сессии, новые сообщения постоянно её прерывали, и p95 дошёл до 29,5 минуты. С промптом с приоритетом делегирования модель делегировала в 4 прогонах из 5. Время выполнения несколько сократилось, но в целом осталось больше, чем у суперагента. В оставшемся прогоне модель сочла работу «мелкой» и сделала её сама, отсюда длинный фиолетовый хвост на графике. Одна сессия вынуждена полагаться на промпт-инжиниринг и сиюминутное суждение модели, чтобы решить, делегировать ли. Двухуровневая архитектура смягчает эту проблему на уровне системы: маршрутизатор сам работу не выполняет, а каждый исполнитель получает только сообщения из тредов, на которые подписан.

Суперагент поддерживает несколько независимых сессий, поэтому за прогон расходует больше токенов, прочитанных из кэша (cache-read tokens), чем архитектуры с одной сессией: токены

обмениваются на более короткое и стабильное время выполнения. Суперагент без сквозной передачи выполняет задачи примерно за то же время, что и суперагент, но его маршрутизатор принимает за прогон примерно на 30% больше решений, а общее число токенов, прочитанных из кэша, растёт с 23,2 млн до 31,8 млн, примерно на 37%. Значит, при этой нагрузке сквозная передача не ускоряет и не улучшает выполнение задач, но существенно сокращает расход токенов и затраты на вывод в сессии маршрутизатора.

7.2 Пакетная обработка в слое маршрутизатора

Чтобы выяснить, какую нагрузку выдерживает одна долгоживущая сессия маршрутизатора с пакетной обработкой из раздела 2.1, мы провели нагрузочный тест одного слоя маршрутизатора. Маршрутизатор вызывал реальную модель, а исполнители использовали модели-заглушки (mock). Мы постепенно повышали частоту поступления триггеров, требующих решения, примерно с 1 до примерно 40 в минуту. На каждой частоте мы измеряли средний размер пакета, время на один запрос и амортизированное время вывода на одно решение. Одно решение может включать несколько запросов к LLM, потому что перед распределением маршрутизатор может рассуждать в несколько шагов или поднимать контекст через вызовы инструментов, поэтому время на решение может превышать время на запрос.



Слева «Размер пакета и время на запрос»: по горизонтали частота поступления триггеров, требующих решения (в минуту); синяя линия: размер пакета B (триггеров на запрос), левая ось; оранжевая пунктирная: время на запрос T (с), правая ось. Справа «Время вывода на решение»: по вертикали время вывода маршрутизатора на одно решение (с), по горизонтали та же частота поступления триггеров.

Как видно на графике выше, когда частота поступления триггеров, требующих решения, растёт примерно с 1 до примерно 40 в минуту, средний размер пакета увеличивается с 1,0 до 2,8. Время на запрос растёт только с 2,5 с до 3,9 с, медленнее размера пакета, поэтому амортизированное время вывода на одно решение падает с 6,4 с до 1,6 с. Иначе говоря, когда внешние события приходят всплесками, слой маршрутизатора повышает эффективную пропускную способность, автоматически объединяя накопившиеся события. Пакетная обработка объясняет и результат из раздела 7.1: без сквозной передачи пиковая нагрузка на маршрутизатор выросла, но один маршрутизатор всё равно не упёрся в предел пропускной способности. Поэтому при нашем нынешнем размере команды нет нужды преждевременно шардировать слой маршрутизатора ценой непрерывности глобального контекста.

8. Открытые проблемы

В разделах 2–6 описаны пять модулей, которые образуют базовую архитектуру суперагента. Эксперименты из раздела 7 показывают, что его ключевые механизмы планирования и маршрутизации работают, когда параллельно выполняется много задач. Система может оставаться относительно простой во многом потому, что на этом этапе мы упростили и инженерную часть, и поддерживаемые сценарии. Помимо этих упрощений, открытыми остаются проблемы в четырёх областях: масштабирование двухуровневой архитектуры (раздел 8.1), работа с агентом без сессий (раздел 8.2), границы контекста и общение агентов между собой (A2A, раздел 8.3), безопасность и приватность (раздел 8.4).

8.1 Как масштабировать двухуровневую архитектуру

Пока большую часть наших повседневных задач при низких инженерных затратах покрывает простое сочетание: одна долгоживущая сессия маршрутизатора, подписка и сквозная передача, никакого прямого общения между исполнителями. Когда задачи становятся крупнее и сильнее связаны, чем может обработать одна сессия маршрутизатора, правильная структура каждого слоя пока не определена. Нам ещё нужно проверить больше сценариев, чтобы понять, когда и насколько мелко шардировать слой маршрутизатора и когда равноправным исполнителям стоит общаться напрямую и в какой топологии совместной работы. Стоит исследовать и совместное проектирование системы и модели. Если обучать модель специально для работы слоя маршрутизатора, то есть оценки событий, пакетного распределения и поднятия контекста, это может ещё повысить пропускную способность слоя маршрутизатора и качество его решений.

Память тоже ограничивает масштабирование системы. Долгоживущий слой маршрутизатора вместе с цепочкой вспоминания по графу ссылок в основном обеспечивает извлечение по запросу недавних задач и истории сессий. Когда система работает месяцы и дольше, главным узким местом становится долговременная память.

8.2 Как работать с агентом без сессий

Двухуровневая архитектура скрывает от пользователей управление несколькими сессиями, поэтому сессии перестают быть главной заботой продуктового дизайна. Но когда нижележащие исполняющие сессии больше не соответствуют напрямую разговорам в интерфейсе, возникает новый вопрос: как показывать задачи пользователям и какие формы

взаимодействия и управления проектами будут ощущаться естественно. Мы думаем, что окно чата стало центром взаимодействия отчасти из-за зависимости от пройденного пути. Больше взаимодействия может переместиться в интерфейсы конкретных предметных областей, как у нашего экземпляра суперагента, который работает прямо в Issues и пул-реквестах GitHub. Сессии могут и остаться в интерфейсе, потому что пользователи к ним привыкли, но один разговор, который видит пользователь, не обязан соответствовать одной нижележащей сессии.

В предыдущих разделах в основном описано взаимодействие с человеком в начале задачи и при приёмке. Но в нынешний переходный период, пока модели быстро улучшаются, многим задачам всё ещё нужен человек на полпути. Стоит исследовать и то, как продуктовый дизайн может позволить пользователям пошагово вмешиваться в ключевые детали, не теряя эффективности автономной фоновой работы. Например, пользователи могли бы в любой момент фонового выполнения вмешаться, перехватить управление или поработать вместе с агентом в реальном времени.

В целом управление проектами и взаимодействие, которые нужны пользователям, сильно зависят от их рабочей области и личного стиля, поэтому итоговый дизайн, скорее всего, будет сильно кастомизирован. У DeepSeek Harness открытая архитектура, и поддерживать такую кастомизацию в нём проще.

8.3 Как определять границы контекста и взаимодействовать через A2A

До сих пор мы обсуждали устройство одного экземпляра суперагента и предполагали, что вся информация в его области собрана в одном глобальном контексте. Когда использование растёт от отдельных людей и небольших команд к крупным

организациям, решение о том, как выделять суперагентов, по сути становится решением о том, какой контекст общий, а какой изолирован. Части организации, которым нужен общий контекст, используют один экземпляр, а части, которым нужна изоляция, получают каждая свой.

Когда внутри организации или между организациями появляются изолированные экземпляры, совместная работа через их границы становится общением агентов между собой (agent-to-agent, A2A), например между личным агентом каждого участника и общим суперагентом команды. Здесь A2A означает общение агентов вообще, а не конкретный протокол. Нам ещё предстоит понять, как агенты, у которых нет общего нижележащего контекста и каждый из которых представляет свою границу прав доступа, могут безопасно и надёжно общаться и работать вместе.

8.4 Как обеспечить безопасность и приватность

Событийная архитектура из раздела 3 позволяет управлять агентом из гораздо большего числа окружений, но и подвергает его большему объёму недоверенного ввода. Если безопасность и приватность не решены хорошо, суперагент может работать только в очень узкой доверенной области.

Например, архитектура из раздела 5 запускает цикл агента в облаке и устанавливает лёгкие демоны на устройства выполнения. Она неявно предполагает, что облако и конечные точки доверяют друг другу. Но облачный цикл агента постоянно принимает внешние события, а конечные устройства хранят приватный код, чувствительные данные и локальные учётные данные. Соединение этих двух сторон создаёт значительные риски для безопасности и приватности. Если взлом на стороне сервера или промпт-инъекция (prompt injection) скомпрометирует цикл агента, под чужой контроль может попасть каждое подключённое к нему конечное устройство.

Учётные данные могут утечь, даже если облако никогда не будет взломано. Пока учётные данные на конечной точке могут попасть в контекст модели, их может раскрыть и обычное выполнение. Поэтому агенту нужен способ пользоваться учётными данными, никогда их не видя. Чтобы эту архитектуру можно было применять шире, она должна решить эти проблемы безопасности и приватности и сохранить способность поднимать контекст из множества разных окружений.

9. Заключение

Эта архитектура не предлагает новую теорию: она применяет классические инженерные идеи компьютерных систем к построению агентных систем. Конкретные реализации будут меняться по мере развития технологий, но мы считаем, что идеи системного проектирования, лежащие в их основе, останутся полезными дольше. Наш внутренний экземпляр суперагента, построенный на этих идеях и на DeepSeek Harness, уже работает непрерывно. Каждый день он помогает воспроизводить и исправлять проблемы, отвечать на ревью кода и разрабатывать сам DeepSeek Harness.

Как мы отметили в начале, большим моделям нужно всё меньше вмешательства человека в отдельных задачах, поэтому узкое место для дальнейшего роста производительности смещается к человеческому вниманию. Двухуровневая архитектура, событийный подход, асинхронное многоканальное взаимодействие, разделение вычислений и хранения и внешний сторожевой агент вместе позволяют системе взять на себя утомительную работу по управлению сессиями и синхронизации контекста. Тогда люди могут подняться выше, от деталей исполнения к решениям более высокого уровня и приёмке.

Агент, построенный так, перестаёт быть только окном чата, которое ждёт инструкций. Он становится автономным соавтором

работы, который постоянно на связи в цифровых окружениях, где работают люди. Для отдельного человека это может быть персональный агент, который помогает с повседневными задачами на разных устройствах. Для команд он становится частью процесса разработки: берёт задачи и продвигает их. Мы считаем, что по мере совместного развития моделей и системных архитектур такой более автономный способ работы может стать нормой во взаимодействии людей и агентов.

Часть описанной в статье архитектуры реализована в dsh-superagent-prototype. Этот плагин задуман лишь как прототип для проверки технического подхода и официальным продуктом DeepSeek не считается.

Автор и благодарности

Статья основана на общей работе команды DeepSeek Harness. Юцзя Цяо (Yujia Qiao) реализовал систему и написал статью. В ревью и обсуждениях участвовали и давали обратную связь Хуаньци Цао (Huanqi Cao), Цзуй Чэнь (Zui Chen), Се-тин Чу (Xieting Chu), Тяньи Цуй (Tianyi Cui), Шиюань Фэн (Shiyuan Feng), Юйдун Хань (Yudong Han), Кэсинь Лю (Kexin Liu), Шичунь Лю (Shichun Liu), Гэсы Ло (Gesi Luo), Ифань Ши (Yifan Shi), Тунжуй Сюн (Tongrui Xiong), Ханьсян Сюй (Hanxiang Xu), С.С. Юй (C.C. Yu), Цзыя Чжан (Ziya Zhang), Ф. Чжоу (F. Zhou) и другие. И система, и эта статья во многом обязаны ежедневному общению в команде и накопленному ею опыту.

Следующий шаг

- База знаний: DeepSeek Harness (dsh): практическое руководство по официальному агентному фреймворку DeepSeek

Если захотите обсудить, как это применить у себя или в команде — пишите в Telegram @pimenov

Прочитать с ИИ

Короткий промпт для резюме, выводов и применения к вашей задаче.

</> Копировать промпт | ChatGPT | Claude

Посмотреть промпт

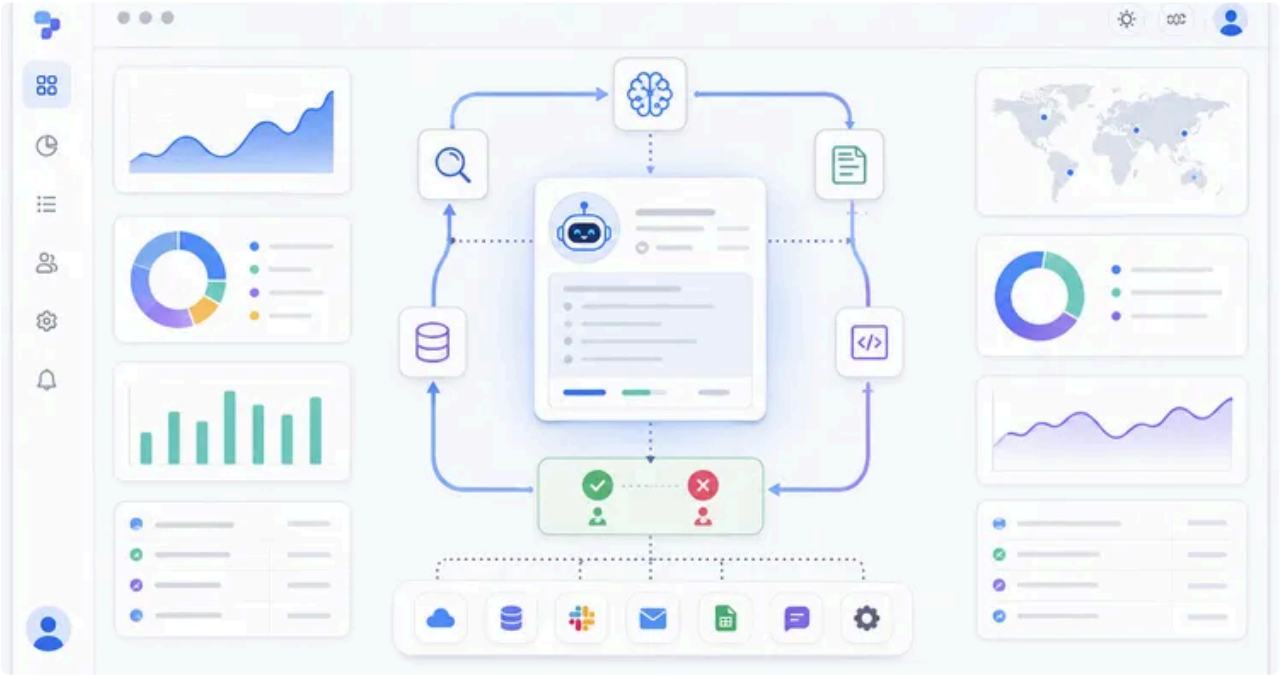
По теме



Блог · 25 сентября 2026

DeepSeek Harness получил desktop-версию: запуск без терминала

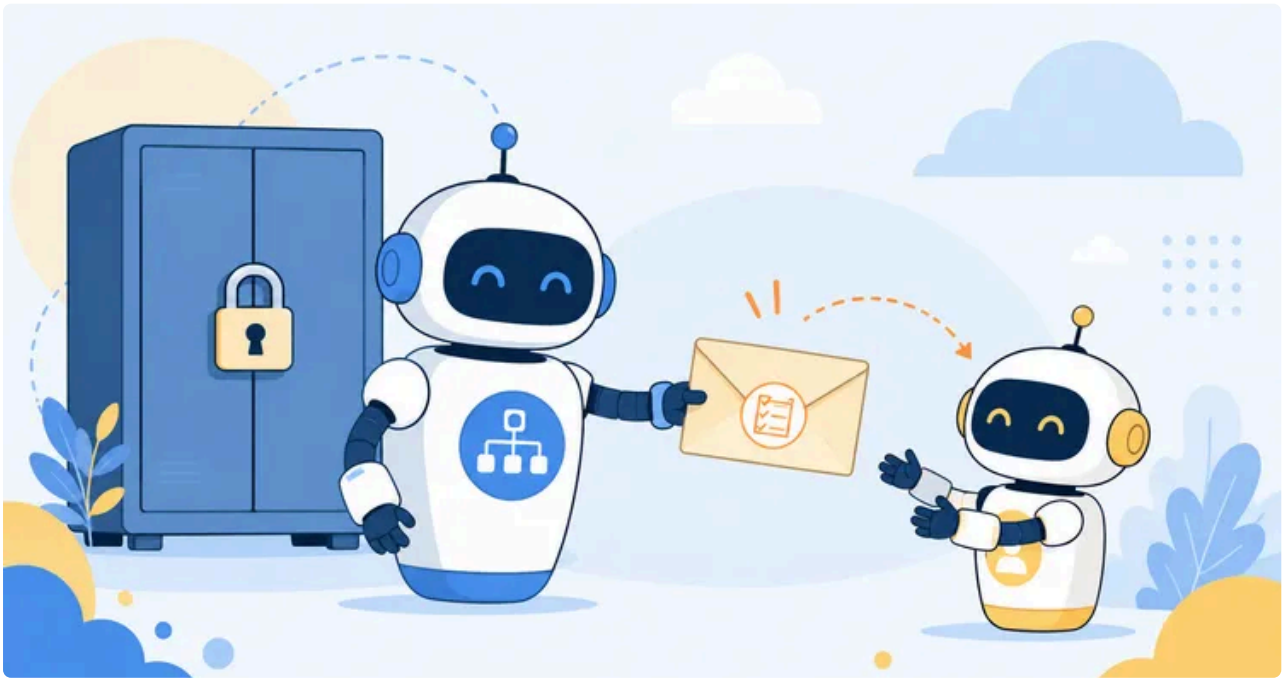
Инфраструктура | Агенты



Статья · 23 августа 2026

Codex становится платформой: агент должен жить там, где уже идёт работа

Агенты | Разработка



База знаний · Обновлено 11 сентября 2026

Четыре принципа делегирования субагентам

Агенты | Безопасность | Инфраструктура